TP2

1 Exercice 1 : Tests unitaires

tests/unit/test_livraison.py

```
import pytest

from app.livraison import frais_livraison


def test_livraison_panier_ordinaire():
    assert frais_livraison(120) == 25


def test_livraison_panier_299():
    assert frais_livraison(299) == 25


def test_livraison_panier_300():
    assert frais_livraison(300) == 0


def test_livraison_panier_301():
    assert frais_livraison(301) == 0


def test_livraison_panier_zero():
    assert frais_livraison(0) == 25


def test_livraison_montant_negatif():
    with pytest.raises(ValueError):
        frais_livraison(-1)
```

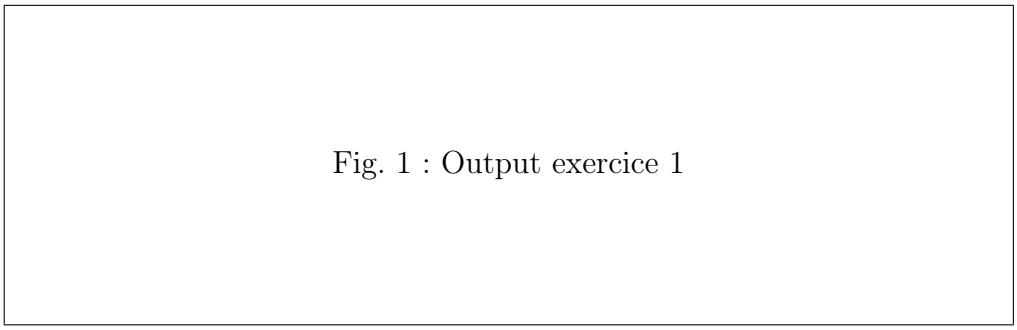


Fig. 1 : Output exercice 1

2 Exercice 2 : Nettoyer un texte avant son analyse

app/texte.py

```
def nettoyer_texte(texte):
  if not isinstance(texte, str):
    raise TypeError("Texte attendu")
  return " ".join(texte.lower().split())
```

tests/test_texte.py

```
import pytest

from app.texte import nettoyer_texte


@pytest.mark.parametrize(
  "entree, attendu",
  [
    ("  BONJOUR   monde  ", "bonjour monde"),
    (" déjà propre ", "déjà propre"),
    ("", ""),
    ("   ", ""),
    (" PYTHON ", "python"),
  ],
)
def test_nettoyer_texte(entree, attendu):
  assert nettoyer_texte(entree) == attendu


@pytest.mark.parametrize("entree", [123, None])
def test_nettoyer_texte_entree_non_textuelle(entree):
  with pytest.raises(TypeError):
    nettoyer_texte(entree)


def test_nettoyer_texte_deja_nettoye_ne_change_pas():
  texte_nettoye = nettoyer_texte("  BONJOUR   monde  ")
  assert nettoyer_texte(texte_nettoye) == texte_nettoye
```

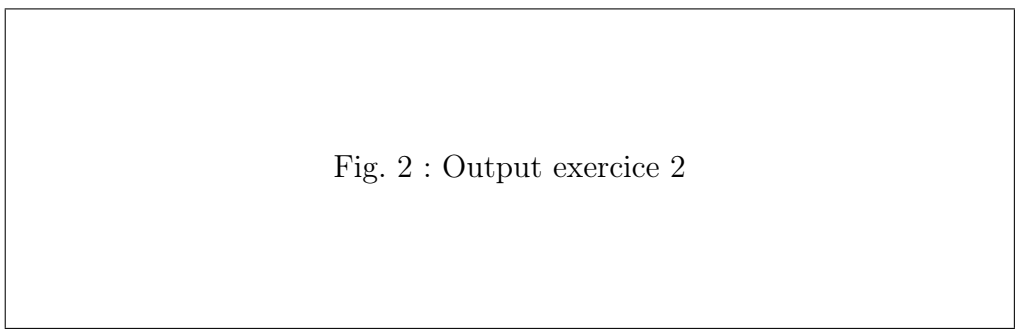


Fig. 2 : Output exercice 2

3 Exercice 3 : Fixtures et tests d'intégration

app/reservation.py

```
class Stock:
  def __init__(self, quantite):
    self.quantite = quantite

  def retirer(self, quantite):
    if quantite <= 0:
      raise ValueError("Quantité positive requise")
    if quantite > self.quantite:
      return False
    self.quantite -= quantite
    return True


class Reservation:
  def __init__(self, stock):
    self.stock = stock

  def reserver(self, quantite):
    if self.stock.retirer(quantite):
      return "confirmee"
    return "refusee"
```

tests/test_reservation.py

```
import pytest

from app.reservation import Reservation, Stock


@pytest.fixture
def stock():
  return Stock(5)


def test_reservation_deux_unites_confirmee_et_stock_reduit(stock):
  reservation = Reservation(stock)
  assert reservation.reserver(2) == "confirmee"
  assert stock.quantite == 3


def test_reservation_six_unites_refusee_et_stock_inchange(stock):
  reservation = Reservation(stock)
  assert reservation.reserver(6) == "refusee"
  assert stock.quantite == 5


def test_reservation_cinq_unites_confirmee_et_stock_vide(stock):
  reservation = Reservation(stock)
  assert reservation.reserver(5) == "confirmee"
  assert stock.quantite == 0
```

```
def test_reservation_zero_unite_leve_value_error_et_stock_inchange(
    stock):
  reservation = Reservation(stock)
  with pytest.raises(ValueError):
    reservation.reserver(0)
  assert stock.quantite == 5
```

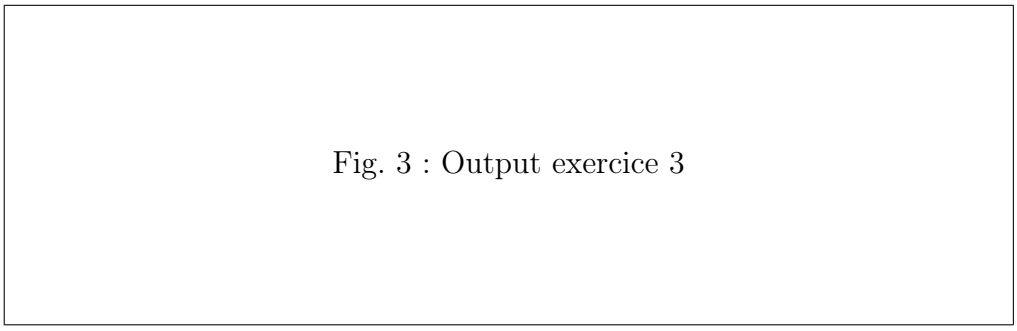


Fig. 3 : Output exercice 3

4 Exercice 4 : Isolation des dépendances

app/moderation.py

```
def decider(modele, message):
  classe = modele.predict(message)
  if classe == "spam":
    return "bloquer"
  return "autoriser"
```

tests/test_moderation.py

```
from unittest.mock import Mock

import pytest

from app.moderation import decider


def test_decider_bloque_un_spam():
  modele = Mock()
  modele.predict.return_value = "spam"
  assert decider(modele, "Offre miracle") == "bloquer"
  modele.predict.assert_called_once_with("Offre miracle")


def test_decider_autorise_un_message_normal():
  modele = Mock()
  modele.predict.return_value = "normal"
  assert decider(modele, "Bonjour") == "autoriser"
  modele.predict.assert_called_once_with("Bonjour")
```

```
def test_decider_propage_erreur_du_modele():
  modele = Mock()
  modele.predict.side_effect = RuntimeError("Modele indisponible")
  with pytest.raises(RuntimeError, match="Modele indisponible"):
    decider(modele, "Offre miracle")
```

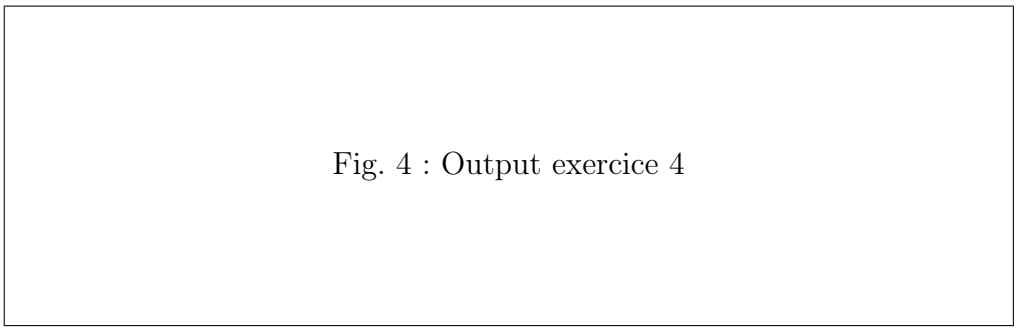


Fig. 4 : Output exercice 4